

credential_leak_audit

challenges/security/ credential_leak_audit.sh

Revision: 1 **Last modified:** 2026-06-10T00:00:00Z

Overview

challenges/security/credential_leak_audit.sh is the §11.4.10 / §11.4.10.A **pre-store credential-leak audit** made standing. Before any secret is stored (and on every CI run), it proves the tracked tree is free of committed secrets and fixed-key credential material. It is read-only, takes no arguments, and never prints a matched secret VALUE (only <path>: <reason>), per §11.4.10 (test scripts MUST NEVER print or log credentials).

It is the **extension-aware companion** to challenges/scripts/credential_leak_grep_challenge.sh (Go-backend focused): this script adds the BobaLink fixed-key-crypto checks that prove the extension is delegate-by-default (every secret is encrypted only under a USER-SUPPLIED session passphrase, never a fixed/empty key).

Prerequisites

- bash, git, grep (ERE) on PATH.
- Run from a git work tree (the script cds to the repo root itself).
- No network, no credentials, no build.

Usage

challenges/security/credential_leak_audit.sh

Exit codes: 0 = clean (CREDENTIAL-LEAK-AUDIT: PASS) · 1 = one or more findings (CREDENTIAL-LEAK-AUDIT: FAIL + a findings list).

What it checks (5)

1. **Tracked secret files** — no `.env` / `*.token` / `*.secret.json` appears in `git ls-files` (whole repo). `.env.example` placeholders are explicitly allowed.
2. **Fixed-passphrase literal** — the reference's removed `"bobalink-extension"` fixed passphrase is ABSENT from owned production source.
3. **Fixed-key crypto** — no `decrypt(..., "literal")` / `encrypt(..., '')` call with a string-literal or empty passphrase in owned production source. The safe, expected form is a runtime variable (`decrypt(bundle, passphrase)`).
4. **AES-key / Bearer literal** — no hardcoded AES/encryption key (≥ 32 hex) or `Bearer <token>` literal in owned production source.
5. **BOBA_API_TOKEN value** — no `BOBA_API_TOKEN=<concrete-value>` committed (placeholders such as `empty`, `<...>`, `your-...`, `changeme`, `example` are allowed).

Scope rationale (why it is not a tree-wide grep)

Check 1 (tracked secret files) spans the WHOLE repo. The **source-code** checks (2-5) are scoped to **owned production source** — `extension/src/`, `download-proxy/src/`, `qBitTorrent-go/` (excluding `*_test.go`), `plugins/*.py`, `webui-bridge.py`. This is deliberate and anti-bluff, not a cover-up:

- `docs/` legitimately **describe** the reference's removed fixed-key defect.
- `extension/tests/security/**` legitimately **assert** the literal is ABSENT and therefore must name it.
- `docs/research/**` is vendored reference material, not Boba's shipped code.
- The crypto module's JSDoc `@example` blocks show `encrypt("plain", "user-passphrase")` in comments — `comment /
// / * / #` lines are excluded so only real executable call sites are flagged.

A real regression that re-introduces a fixed passphrase, a fixed-key crypto call, an AES/Bearer literal, or a concrete `BOBA_API_TOKEN` into production source still FAILs the audit.

Edge cases

- **No matches** — `grep` exit 1 is tolerated (`|| true`); the absence of a finding is the PASS signal, never an error.
- **Binary files** — skipped (`grep -I`).
- **NUL-safe** — tracked paths are iterated via `git ls-files -z` so paths with spaces/newlines are handled.

Anti-bluff (§11.4 / CONST-XII)

The audit greps the REAL tracked tree (real evidence, not metadata). It is proven non-tautological: injecting a fixed-passphrase literal, a fixed-key crypto call, an AES key, and a Bearer literal into a tracked extension/src/ probe file makes the audit report all four findings and exit 1; removing the probe restores PASS.

Related scripts

- challenges/scripts/credential_leak_grep_challenge.sh — Go-backend credential grep gate (sibling).
- extension/tests/security/no-hardcoded-secret.test.ts — Vitest assertion that the fixed passphrase is absent from extension source.
- extension/tests/security/secret-storage.test.ts — proves the token is stored encrypted under a user passphrase.

Cross-references

- constitution/Constitution.md §11.4.10 (credentials handling), §11.4.10.A (pre-store credential-leak audit), §11.4.18 (script documentation mandate).

Last verified: 2026-06-10.